

236. 二叉树的最近公共祖先

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 树, 深度优先搜索, 二叉树
- **Companies:** (Not Available)

给定一个二叉树，找到该树中两个指定节点的最近公共祖先。

百度百科中最近公共祖先的定义为：“对于有根树 T 的两个节点 p 、 q ，最近公共祖先表示为一个节点 x ，满足 x 是 p 、 q 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）。”

示例 1:

输入：root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

输出：3

解释：节点 5 和节点 1 的最近公共祖先是节点 3 。

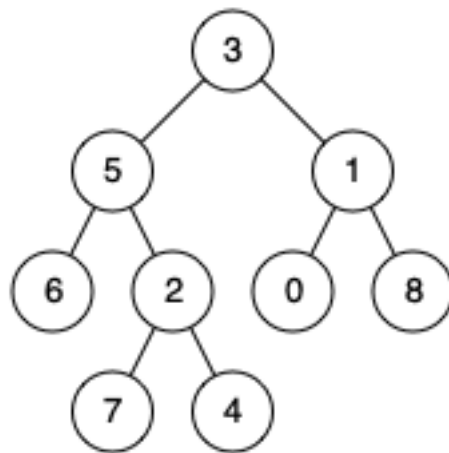


Figure 1: img

示例 2:

输入：root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4

输出：5

解释：节点 5 和节点 4 的最近公共祖先是节点 5 。因为根据定义最近公共祖先节点可以为节点本身。

示例 3:

输入: root = [1,2], p = 1, q = 2

输出: 1

提示:

- 树中节点数目在范围 $[2, 10^5]$ 内。
- $-10^9 \leq \text{Node.val} \leq 10^9$
- 所有 `Node.val` 互不相同。
- $p \neq q$
- p 和 q 均存在于给定的二叉树中。

Discussion | Solution

解决方法

1. 问题理解

目标: 在给定的二叉树 (**不一定是二叉搜索树**) 中, 找到两个指定节点 p 和 q 的最近公共祖先 (Lowest Common Ancestor, LCA)。**LCA 定义:** 节点 x 是 p 和 q 的公共祖先, 且 x 的深度尽可能大。一个节点可以是自己的祖先。**保证:** p 和 q 均存在于树中, 且 $p \neq q$ 。

2. 思路分析

递归 (深度优先搜索 - DFS) 这是一个经典的可以用递归解决的问题。我们考虑对于当前节点 $root$, p 和 q 与它的关系有以下几种情况:

1. p 和 q 分别位于 $root$ 的左右子树中: 那么 $root$ 就是 LCA。
2. p 和 q 都位于 $root$ 的左子树中: 那么 LCA 也在左子树中, 需要递归到左子树查找。
3. p 和 q 都位于 $root$ 的右子树中: 那么 LCA 也在右子树中, 需要递归到右子树查找。
4. p 或 q 其中一个就是 $root$: 根据定义, 该节点 $root$ 就是 LCA (因为另一个节点必然在 $root$ 的子树中)。

基于以上分析, 我们可以设计一个递归函数 `findLCA(node, p, q)`, 其含义是: **在以 $node$ 为根的子树中查找 p 和 q , 并返回它们的 LCA。**

递归函数的返回值定义: - 如果在以 $node$ 为根的子树中找到了 p 或 q 中的一个, 则返回那个找到的节点 (p 或 q)。- 如果在以 $node$ 为根的子树中同时找到了 p 和 q , 则返回它们的 LCA。- 如果在以 $node$ 为根的子树中 p 和 q 都没找到, 则返回 `null`。

递归逻辑:

1. **基本情况 (Base Case):**
 - 如果 $node$ 为 `null`, 说明在这条路径上没找到 p 或 q , 返回 `null`。
 - 如果 $node == p$ 或 $node == q$, 说明找到了 p 或 q 中的一个, 根据定义, 它可能是 LCA (如果另一个在它的子树中), 或者只是 p 或 q 本身。直接返回 $node$ 。
2. **递归步骤 (Recursive Step):**
 - 递归查找左子树: `left_lca = findLCA(node.left, p, q)`。

- 递归查找右子树: `right_lca = findLCA(node.right, p, q)`。

3. 处理结果:

- 如果 `left_lca` 和 `right_lca` 都不为 `null`: 这意味着 `p` 和 `q` 分别位于 `node` 的左右子树中, 因此当前节点 `node` 就是它们的 LCA。返回 `node`。
- 如果 `left_lca` 不为 `null`, 而 `right_lca` 为 `null`: 这意味着 `p` 和 `q` (或者它们的 LCA) 都在左子树中。返回 `left_lca`。
- 如果 `right_lca` 不为 `null`, 而 `left_lca` 为 `null`: 这意味着 `p` 和 `q` (或者它们的 LCA) 都在右子树中。返回 `right_lca`。
- 如果 `left_lca` 和 `right_lca` 都为 `null`: 这意味着 `p` 和 `q` 都不在以 `node` 为根的子树中。返回 `null`。

简化处理结果: 观察上述处理逻辑, 可以简化为: - 如果 `left_lca` 和 `right_lca` 都不为空, 返回 `node`。- 否则, 如果 `left_lca` 不为空, 返回 `left_lca`。- 否则 (即 `left_lca` 为空), 返回 `right_lca` (无论 `right_lca` 是否为空)。这等价于: - 如果 `left_lca` 和 `right_lca` 都不为空, 返回 `node`。- 否则, 返回 `left_lca` 和 `right_lca` 中不为空的那个 (如果都为空则返回 `null`)。

主函数调用: 直接调用 `findLCA(root, p, q)`。

执行流程示例 (`root = [3,5,1,6,2,0,8,null,null,7,4]`, `p = 5`, `q = 1`):

1. `findLCA(3, 5, 1)`
2. `-> left = findLCA(5, 5, 1)`
3. `-> node == p (5)`, 返回 5. (`left = 5`)
4. `-> right = findLCA(1, 5, 1)`
5. `-> node == q (1)`, 返回 1. (`right = 1`)
6. 节点 3: `left (5)` 非空, `right (1)` 非空。返回 `node (3)`。
7. 最终结果: 3

执行流程示例 (`root = [3,5,1,6,2,0,8,null,null,7,4]`, `p = 5`, `q = 4`):

1. `findLCA(3, 5, 4)`
2. `-> left = findLCA(5, 5, 4)`
3. `-> node == p (5)`, 返回 5. (`left = 5`)
4. `-> right = findLCA(1, 5, 4)`
5. `-> left = findLCA(0, 5, 4) -> 返回 null`
6. `-> right = findLCA(8, 5, 4) -> 返回 null`
7. `-> 节点 1: left null, right null. 返回 null. (right = null)`
8. 节点 3: `left (5)` 非空, `right (null)` 为空。返回 `left (5)`。
9. 最终结果: 5 (这里有个小错误, 示例中 4 在 2 的右边, 应该在 5 的子树里)

修正示例 2 的流程 (`root = [3,5,1,6,2,0,8,null,null,7,4]`, `p = 5`, `q = 4`):

1. `findLCA(3, 5, 4)`
2. `-> left = findLCA(5, 5, 4)`
3. `-> node == p (5)`, 返回 5. (`left = 5`) (这里可以优化, 如果找到一个节点是 `p` 或 `q`, 另一个节点肯定在它的子树里, 可以直接返回这个节点。但为了通用性, 我们继续递归)
 - `left_sub = findLCA(6, 5, 4) -> 返回 null`
 - `right_sub = findLCA(2, 5, 4)`
 - `-> left_sub_sub = findLCA(7, 5, 4) -> 返回 null`

- -> right_sub_sub = findLCA(4, 5, 4) -> node == q (4), 返回 4.
- -> 节点 2: left null, right (4) 非空. 返回 4. (right_sub = 4)
- 节点 5: left_sub null, right_sub (4) 非空. 返回 4. (这里之前的理解有误, 应该返回 4)
- **修正递归函数定义和返回值含义:** 递归函数 dfs(node, p, q) 返回在以 node 为根的子树中是否包含 p 或 q. 或者, 更符合 LCA 定义的递归: dfs(node, p, q) 返回在以 node 为根的子树中找到的 p 或 q 或它们的 LCA.

重新梳理递归逻辑 (最常用版本): dfs(node, p, q):

1. Base Case: node is null, return null. node is p or q, return node.
2. left = dfs(node.left, p, q)
3. right = dfs(node.right, p, q)
4. If left and right are both non-null, node is the LCA, return node.
5. If left is non-null, return left.
6. If right is non-null, return right.
7. If both are null, return null. This can be simplified: return node if left and right are non-null, else return left if left else right.

再次执行示例 2 (root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4):

1. dfs(3, 5, 4)
2. -> left = dfs(5, 5, 4)
3. -> Base Case: node == p (5). 返回 5. (left = 5)
4. -> right = dfs(1, 5, 4)
5. -> left_sub = dfs(0, 5, 4) -> Base Case: null. 返回 null.
6. -> right_sub = dfs(8, 5, 4) -> Base Case: null. 返回 null.
7. -> 节点 1: left_sub null, right_sub null. 返回 null. (right = null)
8. 节点 3: left (5) 非空, right (null) 为空. 返回 left (5).
9. 最终结果: 5. (这次对了, 因为当递归到 5 时, 它直接返回了 5, 没有继续往下找 4)

方法二: 存储父节点 + 向上查找

1. **遍历树存储父指针:** 使用 DFS 或 BFS 遍历整棵树, 用一个哈希表 parent_map 存储每个节点与其父节点的映射关系 ({child: parent}).
2. **记录 p 的祖先:** 从节点 p 开始, 沿着父指针向上遍历, 将所有祖先 (包括 p 自身) 存储在一个集合 ancestors_p 中.
3. **查找 q 的祖先:** 从节点 q 开始, 沿着父指针向上遍历. 对于遍历到的每个节点 curr:
 - 检查 curr 是否在 ancestors_p 集合中.
 - 如果存在, 那么 curr 就是第一个公共祖先, 即 LCA. 返回 curr.
4. **边界情况:** 由于题目保证 p 和 q 存在且不相等, 并且根节点是所有节点的祖先, 此方法一定能找到 LCA.

缺点: 需要额外 $O(N)$ 空间存储父指针和 $O(H)$ 空间存储祖先集合 (最坏 $O(N)$). 需要两次遍历 (一次建图, 一次查找).

3. 代码实现 (Python)

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class TreeNode:
    def __init__(self, x):
        self.val = x
        self.left = None
        self.right = None

class Solution:
    # 方法一: 递归 (DFS)
    def lowestCommonAncestorRecursive(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode')
        """
        使用递归查找最近公共祖先
        """
        # Base Case 1: 节点为空
        if not root:
            return None
        # Base Case 2: 找到 p 或 q
        if root == p or root == q:
            return root

        # 递归查找左右子树
        left_lca = self.lowestCommonAncestorRecursive(root.left, p, q)
        right_lca = self.lowestCommonAncestorRecursive(root.right, p, q)

        # 处理结果
        # 1. p 和 q 分别在左右子树
        if left_lca and right_lca:
            return root
        # 2. p 和 q 都在左子树 (或其中一个是 root.left 且另一个在更深层)
        if left_lca:
            return left_lca
        # 3. p 和 q 都在右子树 (或其中一个是 root.right 且另一个在更深层)
        if right_lca:
            return right_lca
        # 4. p 和 q 都不在以 root 为根的子树中
        return None

    # 简化版处理结果:
```

```

        # if left_lca and right_lca:
        #     return root
        # return left_lca if left_lca else right_lca

# 方法二: 存储父节点
def lowestCommonAncestorParentMap(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode')
    """
    使用存储父节点的方法查找最近公共祖先
    """
    parent_map = {root: None}
    queue = collections.deque([root]) # BFS 建立 parent_map

    # BFS 建立父指针映射, 直到 p 和 q 都找到为止 (优化)
    found_p = False
    found_q = False
    while queue and not (found_p and found_q):
        node = queue.popleft()
        if node == p: found_p = True
        if node == q: found_q = True

        if node.left:
            parent_map[node.left] = node
            queue.append(node.left)
        if node.right:
            parent_map[node.right] = node
            queue.append(node.right)

    # 记录 p 的所有祖先
    ancestors_p = set()
    curr = p
    while curr:
        ancestors_p.add(curr)
        curr = parent_map[curr]

    # 查找 q 的祖先, 第一个在 p 祖先集合中的即为 LCA
    curr = q
    while curr:
        if curr in ancestors_p:
            return curr
        curr = parent_map[curr]

    return None # 理论上不会执行到

# 最终提交选择递归方法
def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
    # 需要导入 collections

```

```
import collections
# return self.lowestCommonAncestorParentMap(root, p, q)
return self.lowestCommonAncestorRecursive(root, p, q)
```

4. 复杂度分析

- **方法一（递归）：**
 - 时间复杂度： $O(N)$ ，其中 N 是树中的节点数。最坏情况下需要访问所有节点。
 - 空间复杂度： $O(H)$ ，其中 H 是树的高度。空间消耗主要来自递归调用栈。最坏情况下（链状树）为 $O(N)$ ，平均情况下（平衡树）为 $O(\log N)$ 。
- **方法二（存储父节点）：**
 - 时间复杂度： $O(N)$ 。建立父指针映射需要 $O(N)$ ，向上查找祖先最坏需要 $O(H)$ 。总体为 $O(N)$ 。
 - 空间复杂度： $O(N)$ 。`parent_map` 需要 $O(N)$ 空间，`ancestors_p` 集合最坏需要 $O(H)$ （链状树 $O(N)$ ）空间，BFS 队列最坏需要 $O(W)$ 空间。总体为 $O(N)$ 。

结论：递归方法在空间复杂度上通常更优（平均 $O(\log N)$ vs $O(N)$ ），且代码更简洁。

5. 如何在面试中讲解

1. **明确 LCA 定义：**
 - “最近公共祖先是 p 和 q 的所有共同祖先中，深度最大的那个。节点可以是自己的祖先。”
2. **核心思路（递归）：**
 - “考虑当前节点 $root$ 和 p, q 的关系。我们可以设计一个递归函数 `dfs(node, p, q)`，它在以 $node$ 为根的子树中查找 p 和 q ，并返回它们的 LCA（如果都在子树中）、或者返回找到的 p 或 q （如果只找到一个）、或者返回 `null`（如果都没找到）。”
3. **递归函数逻辑：**
 - “Base Cases:
 - $node$ 为空，返回 `null`。
 - $node$ 是 p 或 q ，返回 $node$ 。”
 - “递归调用:
 - `left = dfs(node.left, p, q)`
 - `right = dfs(node.right, p, q)`”
 - “合并结果:
 - 如果 `left` 和 `right` 都非空，说明 p 和 q 分居左右子树，当前 $node$ 就是 LCA，返回 $node$ 。
 - 如果只有 `left` 非空，说明 p 和 q 都在左子树（或其中一个是 `node.left`），LCA 在左边，返回 `left`。
 - 如果只有 `right` 非空，同理，返回 `right`。
 - 如果 `left` 和 `right` 都为空，说明 p 和 q 都不在此子树，返回 `null`。”
 - “可以简化为：如果 `left` 和 `right` 都非空，返回 $node$ ；否则返回 `left` 或 `right` 中非空的那个（或 `null`）。”
4. **复杂度分析：**
 - “时间 $O(N)$ ，每个节点访问常数。”
 - “空间 $O(H)$ ，递归栈深度。”

5. 备选思路 (存储父节点):

- “也可以先遍历一次树，用哈希表记录每个节点的父节点。”
- “然后从 p 出发，记录所有祖先到一个集合。”
- “再从 q 出发，向上查找，第一个出现在 p 祖先集中的节点就是 LCA。”
- “复杂度：时间 $O(N)$ ，空间 $O(N)$ 。递归方法通常更优。”

6. 注意点:

- 题目保证 p, q 存在且不同。
- 节点值唯一。
- 这是普通二叉树，不是 BST。